

Sentiment Analysis on Amazon reviews

```
In [1]: import warnings
warnings.filterwarnings('ignore')
```

```
In [2]: #Basic Libraries
import pandas as pd
import numpy as np

#NLTK Libraries
import nltk
import re
import string
from wordcloud import WordCloud, STOPWORDS
from nltk.stem.porter import PorterStemmer
from sklearn.feature_extraction.text import TfidfVectorizer

# Machine Learning Libraries
import sklearn
from sklearn.svm import SVC
from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import MinMaxScaler
from sklearn.ensemble import ExtraTreesClassifier
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import GridSearchCV
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.naive_bayes import BernoulliNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.multiclass import OneVsRestClassifier
from sklearn.svm import SVC
from sklearn.pipeline import Pipeline
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import label_binarize
from sklearn import svm, datasets
from sklearn import preprocessing

#Metrics Libraries
from sklearn import metrics
from sklearn.metrics import classification_report
from sklearn.model_selection import cross_val_score
from sklearn.metrics import roc_auc_score
from sklearn.metrics import roc_curve, auc

#Visualization Libraries
import matplotlib.pyplot as plt
from matplotlib import rcParams
import seaborn as sns
from textblob import TextBlob
from plotly import tools
import plotly.graph_objs as go
from plotly.offline import iplot
%matplotlib inline

#Ignore warnings
import warnings
warnings.filterwarnings('ignore')

#Other miscellaneous Libraries
from scipy import interp
from itertools import cycle
import cufflinks as cf
from collections import defaultdict
from collections import Counter
from imblearn.over_sampling import SMOTE
```



```
In [3]: raw_reviews = pd.read_csv(r"C:\Users\JANHAVI\Downloads\Musical_instruments_reviews.csv")
## print shape of dataset with rows and columns and information
print ("The shape of the data is (row, column):"+ str(raw_reviews.shape))
print (raw_reviews.info())
```

```
The shape of the data is (row, column):(10261, 9)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10261 entries, 0 to 10260
Data columns (total 9 columns):
#   Column                Non-Null Count  Dtype
---  -
0   reviewerID            10261 non-null  object
1   asin                  10261 non-null  object
2   reviewerName          10234 non-null  object
3   helpful               10261 non-null  object
4   reviewText            10254 non-null  object
5   overall               10261 non-null  float64
6   summary               10261 non-null  object
7   unixReviewTime        10261 non-null  int64
8   reviewTime            10261 non-null  object
dtypes: float64(1), int64(1), object(7)
memory usage: 721.6+ KB
None
```

```
In [4]: raw_reviews.head()
```

Out[4]:

	reviewerID	asin	reviewerName	helpful	reviewText	overall	summary	unixReviewTime	reviewTime
0	A2IBPI20UZIR0U	1384719342	cassandra tu "Yeah, well, that's just like, u...	[0, 0]	Not much to write about here, but it does exac...	5.0	good	1393545600	02 28, 2014
1	A14VAT5EAX3D9S	1384719342	Jake	[13, 14]	The product does exactly as it should and is q...	5.0	Jake	1363392000	03 16, 2013
2	A195EZSQDW3E21	1384719342	Rick Bennette "Rick Bennette"	[1, 1]	The primary job of this device is to block the...	5.0	It Does The Job Well	1377648000	08 28, 2013
3	A2C00NNG1ZQQG2	1384719342	RustyBill "Sunday Rocker"	[0, 0]	Nice windscreen protects my MXL mic and preven...	5.0	GOOD WINDSCREEN FOR THE MONEY	1392336000	02 14, 2014
4	A94QU4C90B1AX	1384719342	SEAN MASLANKA	[0, 0]	This pop filter is great. It looks and perform...	5.0	No more pops when I record my vocals.	1392940800	02 21, 2014

```
In [5]: #Creating a copy
process_reviews=raw_reviews.copy()

#Checking for null values
process_reviews.isnull().sum()
```

```
Out[5]: reviewerID      0
asin                  0
reviewerName         27
helpful              0
reviewText           7
overall              0
summary              0
unixReviewTime       0
reviewTime           0
dtype: int64
```

```
In [6]: process_reviews['reviewText']=process_reviews['reviewText'].fillna('Missing')
```

```
In [7]: process_reviews['reviews']=process_reviews['reviewText']+process_reviews['summary']
process_reviews=process_reviews.drop(['reviewText', 'summary'], axis=1)
process_reviews.head()
```

Out[7]:

	reviewerID	asin	reviewerName	helpful	overall	unixReviewTime	reviewTime	reviews
0	A2IBPI20UZIR0U	1384719342	cassandra tu "Yeah, well, that's just like, u...	[0, 0]	5.0	1393545600	02 28, 2014	Not much to write about here, but it does exac...
1	A14VAT5EAX3D9S	1384719342	Jake	[13, 14]	5.0	1363392000	03 16, 2013	The product does exactly as it should and is q...
2	A195EZSQDW3E21	1384719342	Rick Bennette "Rick Bennette"	[1, 1]	5.0	1377648000	08 28, 2013	The primary job of this device is to block the...
3	A2C00NNG1ZQQG2	1384719342	RustyBill "Sunday Rocker"	[0, 0]	5.0	1392336000	02 14, 2014	Nice windscreen protects my MXL mic and preven...
4	A94QU4C90B1AX	1384719342	SEAN MASLANKA	[0, 0]	5.0	1392940800	02 21, 2014	This pop filter is great. It looks and perform...

```
In [8]: #Figuring out the distribution of categories
process_reviews['overall'].value_counts()
```

```
Out[8]: overall
5.0    6938
4.0    2084
3.0     772
2.0     250
1.0     217
Name: count, dtype: int64
```

```
In [9]: def f(row):

    '''This function returns sentiment value based on the overall ratings from the user'''

    if row['overall'] == 3.0:
        val = 'Neutral'
    elif row['overall'] == 1.0 or row['overall'] == 2.0:
        val = 'Negative'
    elif row['overall'] == 4.0 or row['overall'] == 5.0:
        val = 'Positive'
    else:
        val = -1
    return val
```

```
In [10]: #Applying the function in our new column
process_reviews['sentiment'] = process_reviews.apply(f, axis=1)
process_reviews.head()
```

```
Out[10]:
```

	reviewerID	asin	reviewerName	helpful	overall	unixReviewTime	reviewTime	reviews	sentiment
0	A2IBPI20UZIR0U	1384719342	cassandra tu "Yeah, well, that's just like, u...	[0, 0]	5.0	1393545600	02 28, 2014	Not much to write about here, but it does exac...	Positive
1	A14VAT5EAX3D9S	1384719342	Jake	[13, 14]	5.0	1363392000	03 16, 2013	The product does exactly as it should and is q...	Positive
2	A195EZSQDW3E21	1384719342	Rick Bennette "Rick Bennette"	[1, 1]	5.0	1377648000	08 28, 2013	The primary job of this device is to block the...	Positive
3	A2C00NNG1ZQQG2	1384719342	RustyBill "Sunday Rocker"	[0, 0]	5.0	1392336000	02 14, 2014	Nice windscreen protects my MXL mic and preven...	Positive
4	A94QU4C90B1AX	1384719342	SEAN MASLANKA	[0, 0]	5.0	1392940800	02 21, 2014	This pop filter is great. It looks and perform...	Positive

```
In [11]: process_reviews['sentiment'].value_counts()
```

```
Out[11]: sentiment
Positive    9022
Neutral     772
Negative    467
Name: count, dtype: int64
```

```
In [12]: # new data frame which has date and year
new = process_reviews["reviewTime"].str.split(" ", n = 1, expand = True)

# making separate date column from new data frame
process_reviews["date"] = new[0]

# making separate year column from new data frame
process_reviews["year"] = new[1]

process_reviews = process_reviews.drop(['reviewTime'], axis=1)
process_reviews.head()
```

```
Out[12]:
```

	reviewerID	asin	reviewerName	helpful	overall	unixReviewTime	reviews	sentiment	date	year
0	A2IBPI20UZIR0U	1384719342	cassandra tu "Yeah, well, that's just like, u...	[0, 0]	5.0	1393545600	Not much to write about here, but it does exac...	Positive	02 28	2014
1	A14VAT5EAX3D9S	1384719342	Jake	[13, 14]	5.0	1363392000	The product does exactly as it should and is q...	Positive	03 16	2013
2	A195EZSQDW3E21	1384719342	Rick Bennette "Rick Bennette"	[1, 1]	5.0	1377648000	The primary job of this device is to block the...	Positive	08 28	2013
3	A2C00NNG1ZQQG2	1384719342	RustyBill "Sunday Rocker"	[0, 0]	5.0	1392336000	Nice windscreen protects my MXL mic and preven...	Positive	02 14	2014
4	A94QU4C90B1AX	1384719342	SEAN MASLANKA	[0, 0]	5.0	1392940800	This pop filter is great. It looks and perform...	Positive	02 21	2014

```
In [13]: # Splitting the date
new1 = process_reviews["date"].str.split(" ", n = 1, expand = True)

# adding month to the main dataset
process_reviews["month"] = new1[0]

# adding day to the main dataset
process_reviews["day"] = new1[1]

process_reviews = process_reviews.drop(['date'], axis=1)
process_reviews.head()
```

Out[13]:

	reviewerID	asin	reviewerName	helpful	overall	unixReviewTime	reviews	sentiment	year	month	day
0	A2IBPI20UZIR0U	1384719342	cassandra tu "Yeah, well, that's just like, u...	[0, 0]	5.0	1393545600	Not much to write about here, but it does exac...	Positive	2014	02	28
1	A14VAT5EAX3D9S	1384719342	Jake	[13, 14]	5.0	1363392000	The product does exactly as it should and is q...	Positive	2013	03	
2	A195EZSQDW3E21	1384719342	Rick Bennette "Rick Bennette"	[1, 1]	5.0	1377648000	The primary job of this device is to block the...	Positive	2013	08	28
3	A2C00NNG1ZQQG2	1384719342	RustyBill "Sunday Rocker"	[0, 0]	5.0	1392336000	Nice windscreen protects my MXL mic and preven...	Positive	2014	02	14
4	A94QU4C90B1AX	1384719342	SEAN MASLANKA	[0, 0]	5.0	1392940800	This pop filter is great. It looks and perform...	Positive	2014	02	21

```
In [14]: # Splitting the dataset based on comma and square bracket
new1 = process_reviews["helpful"].str.split(",", n = 1, expand = True)
new2 = new1[0].str.split("[", n = 1, expand = True)
new3 = new1[1].str.split("]", n = 1, expand = True)

#Resetting the index
new2.reset_index(drop=True, inplace=True)
new3.reset_index(drop=True, inplace=True)

#Dropping empty columns due to splitting
new2 = new2.drop([0], axis=1)
new3 = new3.drop([1], axis=1)

#Concatenating the splitted columns
helpful = pd.concat([new2, new3], axis=1)

# I found few spaces in new3, so it is better to strip all the values to find the rate
def trim_all_columns(df):
    """
    Trim whitespace from ends of each value across all series in dataframe
    """
    trim_strings = lambda x: x.strip() if isinstance(x, str) else x
    return df.applymap(trim_strings)

#Applying the function
helpful = trim_all_columns(helpful)

#Converting into integer types
helpful[0] = helpful[0].astype(str).astype(int)
helpful[1] = helpful[1].astype(str).astype(int)

#Dividing the two columns, we have 0 in the second columns when dvided gives error, so I'm ignoring those errors
try:
    helpful['result'] = helpful[1]/helpful[0]
except ZeroDivisionError:
    helpful['result'] = 0

#Filling the NaN values(created due to dividing) with 0
helpful['result'] = helpful['result'].fillna(0)

#Rounding of the results to two decimal places
helpful['result'] = helpful['result'].round(2)

#Attaching the results to a new column of the main dataframe
process_reviews['helpful_rate'] = helpful['result']

#dropping the helpful column from main dataframe
process_reviews = process_reviews.drop(['helpful'], axis=1)
```

In [15]: `process_reviews.head()`

Out[15]:

	reviewerID	asin	reviewerName	overall	unixReviewTime	reviews	sentiment	year	month	day	helpful_rate
0	A2IBPI20UZIR0U	1384719342	cassandra tu "Yeah, well, that's just like, u...	5.0	1393545600	Not much to write about here, but it does exac...	Positive	2014	02	28	0.00
1	A14VAT5EAX3D9S	1384719342	Jake	5.0	1363392000	The product does exactly as it should and is q...	Positive	2013	03	16	0.93
2	A195EZSQDW3E21	1384719342	Rick Bennette "Rick Bennette"	5.0	1377648000	The primary job of this device is to block the...	Positive	2013	08	28	1.00
3	A2C00NNG1ZQQG2	1384719342	RustyBill "Sunday Rocker"	5.0	1392336000	Nice windscreen protects my MXL mic and preven...	Positive	2014	02	14	0.00
4	A94QU4C90B1AX	1384719342	SEAN MASLANKA	5.0	1392940800	This pop filter is great. It looks and perform...	Positive	2014	02	21	0.00

In [16]: `process_reviews['helpful_rate'].value_counts()`

Out[16]: helpful_rate
0.00 7215
1.00 2040
0.50 266
0.67 136
0.75 111
...
0.56 1
0.15 1
0.13 1
0.43 1
0.69 1
Name: count, Length: 65, dtype: int64

In [17]: `#Removing unnecessary columns
process_reviews=process_reviews.drop(['reviewerName','unixReviewTime'], axis=1)
#Creating a copy
clean_reviews=process_reviews.copy()`

In [18]: `def review_cleaning(text):
 '''Make text lowercase, remove text in square brackets,remove links,remove punctuation
 and remove words containing numbers.'''
 text = str(text).lower()
 text = re.sub('\[.*?\]', '', text)
 text = re.sub('https?://\S+|www.\S+', '', text)
 text = re.sub('<.*?>+', '', text)
 text = re.sub('[%s]' % re.escape(string.punctuation), '', text)
 text = re.sub('\n', '', text)
 text = re.sub('\w*\d\w*', '', text)
 return text`

In [19]: `process_reviews['reviews']=process_reviews['reviews'].apply(lambda x:review_cleaning(x))
process_reviews.head()`

Out[19]:

	reviewerID	asin	overall	reviews	sentiment	year	month	day	helpful_rate
0	A2IBPI20UZIR0U	1384719342	5.0	not much to write about here but it does exact...	Positive	2014	02	28	0.00
1	A14VAT5EAX3D9S	1384719342	5.0	the product does exactly as it should and is q...	Positive	2013	03	16	0.93
2	A195EZSQDW3E21	1384719342	5.0	the primary job of this device is to block the...	Positive	2013	08	28	1.00
3	A2C00NNG1ZQQG2	1384719342	5.0	nice windscreen protects my mxl mic and preven...	Positive	2014	02	14	0.00
4	A94QU4C90B1AX	1384719342	5.0	this pop filter is great it looks and performs...	Positive	2014	02	21	0.00

In [20]: `stop_words= ['yourselves', 'between', 'whom', 'itself', 'is', "she's", 'up', 'herself', 'here', 'your', 'each',
 'we', 'he', 'my', "you've", 'having', 'in', 'both', 'for', 'themselves', 'are', 'them', 'other',
 'and', 'an', 'during', 'their', 'can', 'yourself', 'she', 'until', 'so', 'these', 'ours', 'above',
 'what', 'while', 'have', 're', 'more', 'only', "needn't", 'when', 'just', 'that', 'were', "don't",
 'very', 'should', 'any', 'y', 'isn', 'who', 'a', 'they', 'to', 'too', "should've", 'has', 'before',
 'into', 'yours', 'it's", 'do', 'against', 'on', 'now', 'her', 've', 'd', 'by', 'am', 'from',
 'about', 'further', "that'll", "you'd", 'you', 'as', 'how', 'been', 'the', 'or', 'doing', 'such',
 'his', 'himself', 'ourselves', 'was', 'through', 'out', 'below', 'own', 'myself', 'theirs',
 'me', 'why', 'once', 'him', 'than', 'be', 'most', "you'll", 'same', 'some', 'with', 'few', 'it',
 'at', 'after', 'its', 'which', 'there', 'our', 'this', 'hers', 'being', 'did', 'of', 'had', 'under',
 'over', 'again', 'where', 'those', 'then', "you're", 'i', 'because', 'does', 'all']`

```
In [21]: process_reviews['reviews'] = process_reviews['reviews'].apply(lambda x: ' '.join([word for word in x.split() if word not in process_reviews.head()])
```

Out[21]:

	reviewerID	asin	overall	reviews	sentiment	year	month	day	helpful_rate
0	A2IBPI20UZIR0U	1384719342	5.0	not much write but exactly supposed filters po...	Positive	2014	02	28	0.00
1	A14VAT5EAX3D9S	1384719342	5.0	product exactly quite affordablei not realized...	Positive	2013	03	16	0.93
2	A195EZSQDW3E21	1384719342	5.0	primary job device block breath would otherwis...	Positive	2013	08	28	1.00
3	A2C00NNG1ZQQG2	1384719342	5.0	nice windscreen protects mxl mic prevents pops...	Positive	2014	02	14	0.00
4	A94QU4C90B1AX	1384719342	5.0	pop filter great looks performs like studio fi...	Positive	2014	02	21	0.00

```
In [22]: pd.DataFrame(process_reviews.groupby('sentiment')['helpful_rate'].mean())
```

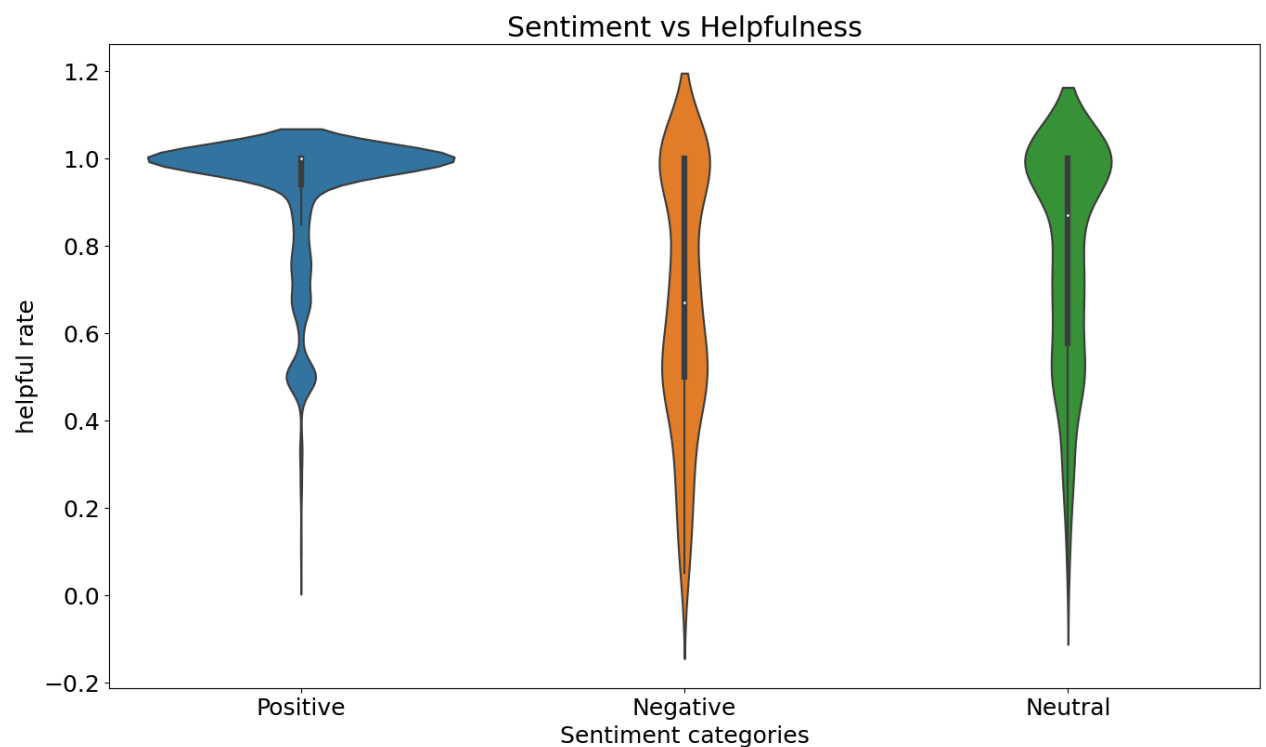
Out[22]:

	helpful_rate
sentiment	
Negative	0.307559
Neutral	0.275687
Positive	0.260505

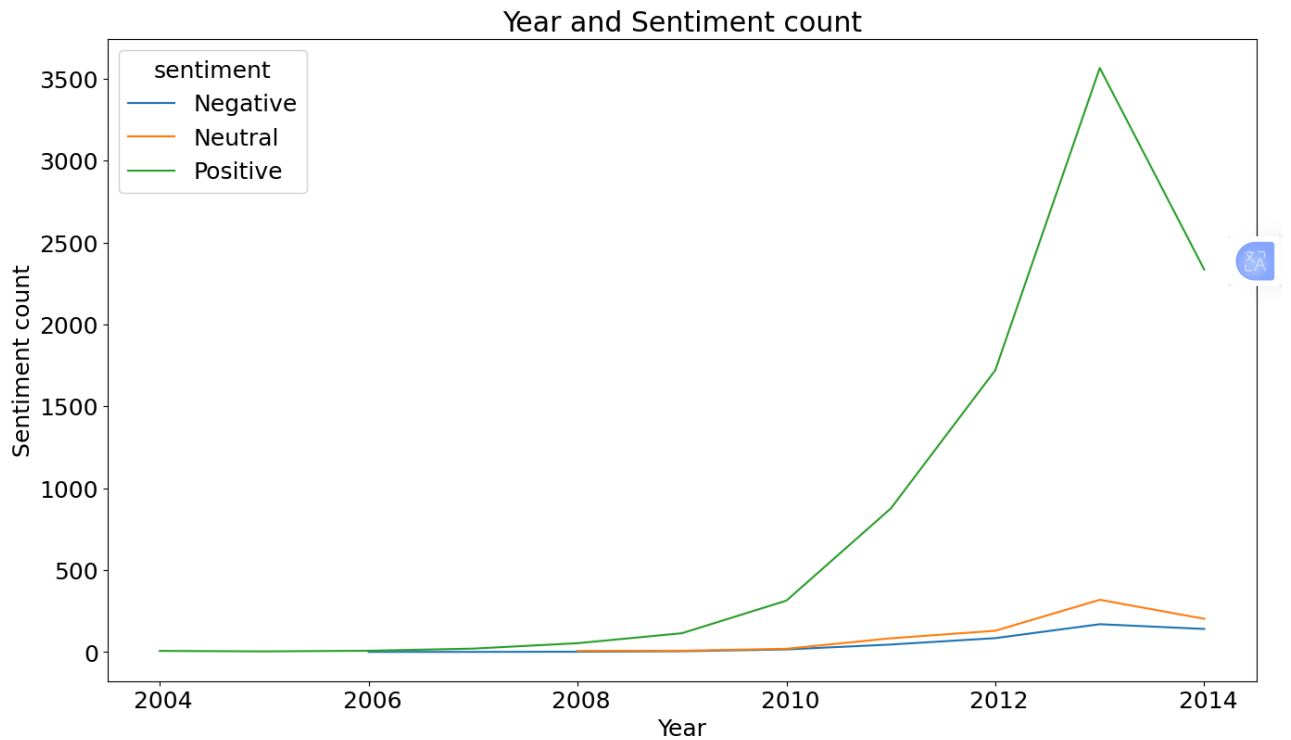
```
In [23]: #Plot Layout
plt.rcParams.update({'font.size': 18})
rcParams['figure.figsize'] = 16,9

# Creating dataframe and removing 0 helpfulrate records
senti_help= pd.DataFrame(process_reviews, columns = ['sentiment', 'helpful_rate'])
senti_help = senti_help[senti_help['helpful_rate'] != 0.00]

#Plotting phase
sns.violinplot( x=senti_help["sentiment"], y=senti_help["helpful_rate"])
plt.title('Sentiment vs Helpfulness')
plt.xlabel('Sentiment categories')
plt.ylabel('helpful rate')
plt.show()
```

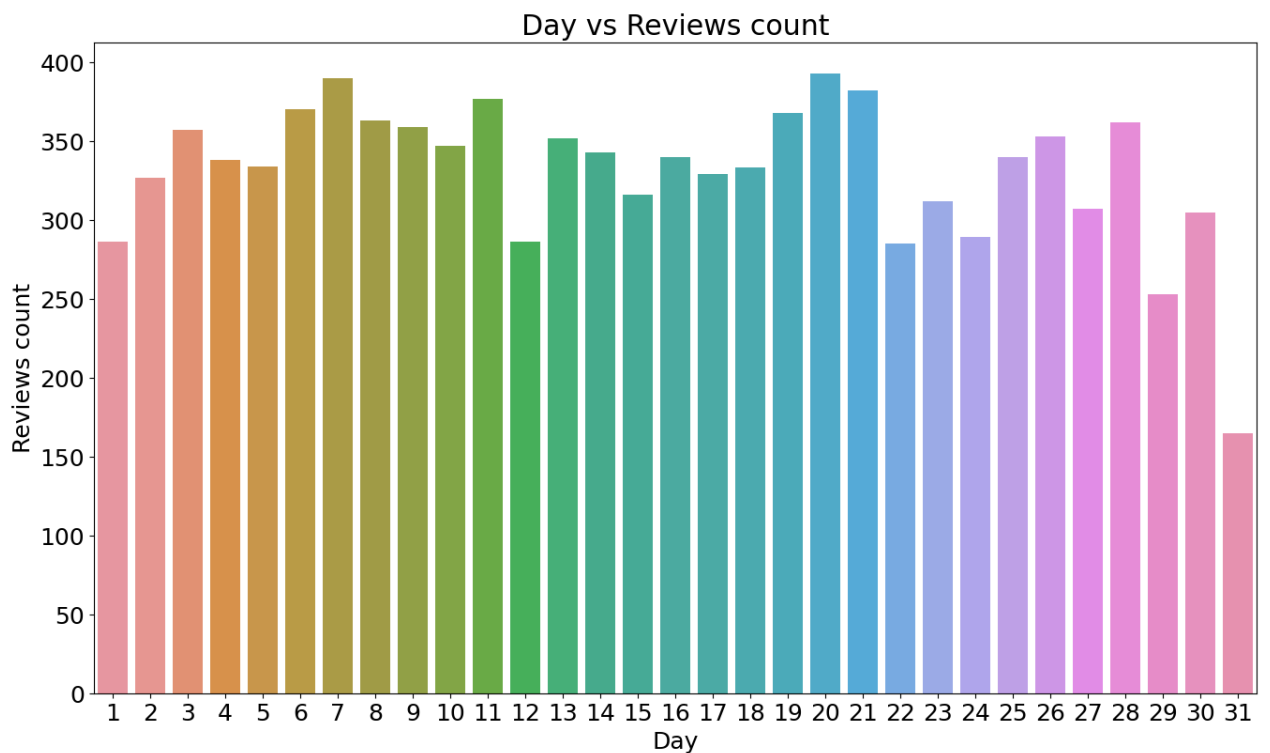


```
In [24]: process_reviews.groupby(['year', 'sentiment'])['sentiment'].count().unstack().plot(legend=True)
plt.title('Year and Sentiment count')
plt.xlabel('Year')
plt.ylabel('Sentiment count')
plt.show()
```



```
In [25]: #Creating a dataframe
day=pd.DataFrame(process_reviews.groupby('day')['reviews'].count()).reset_index()
day['day']=day['day'].astype('int64')
day.sort_values(by=['day'])

#Plotting the graph
sns.barplot(x="day", y="reviews", data=day)
plt.title('Day vs Reviews count')
plt.xlabel('Day')
plt.ylabel('Reviews count')
plt.show()
```



```
In [26]: process_reviews['polarity'] = process_reviews['reviews'].map(lambda text: TextBlob(text).sentiment.polarity)
process_reviews['review_len'] = process_reviews['reviews'].astype(str).apply(len)
process_reviews['word_count'] = process_reviews['reviews'].apply(lambda x: len(str(x).split()))
```

```
In [27]: process_reviews.head()
```

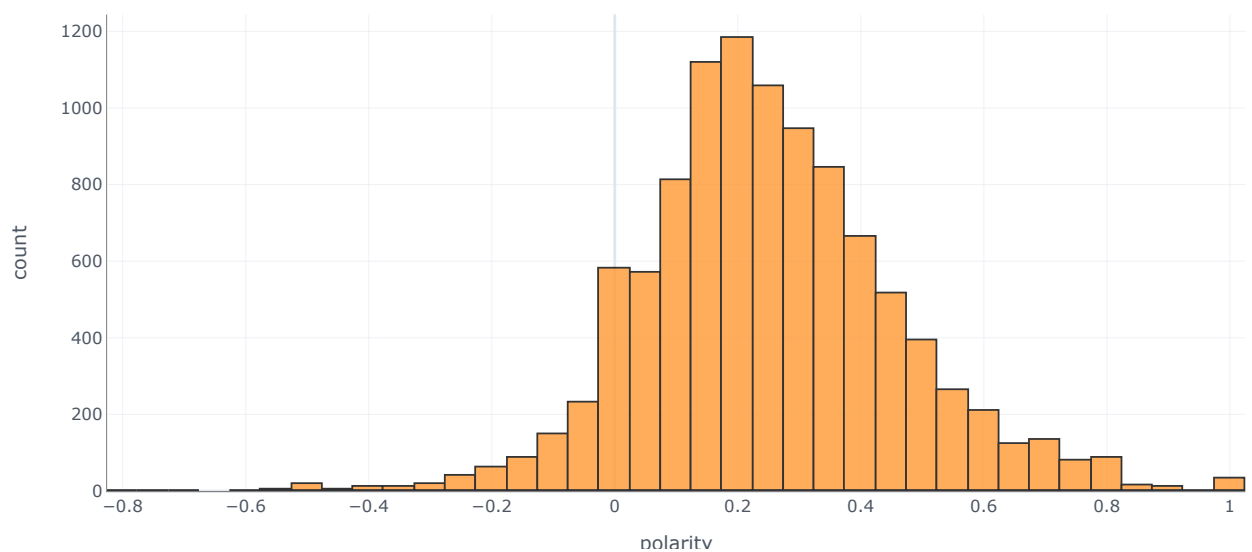
```
Out[27]:
```

	reviewerID	asin	overall	reviews	sentiment	year	month	day	helpful_rate	polarity	review_len	word_count
0	A2IBPI20UZIR0U	1384719342	5.0	not much write but exactly supposed filters po...	Positive	2014	02	28	0.00	0.250000	162	25
1	A14VAT5EAX3D9S	1384719342	5.0	product exactly quite affordable! not realized...	Positive	2013	03	16	0.93	0.014286	356	55
2	A195EZSQDW3E21	1384719342	5.0	primary job device block breath would otherwis...	Positive	2013	08	28	1.00	0.167500	315	48
3	A2C00NNG1ZQQG2	1384719342	5.0	nice windscreen protects mxl mic prevents pops...	Positive	2014	02	14	0.00	0.333333	169	
4	A94QU4C90B1AX	1384719342	5.0	pop filter great looks performs like studio fi...	Positive	2014	02	21	0.00	0.800000	136	21

```
In [28]: cf.go_offline()
cf.set_config_file(offline=False, world_readable=True)
```

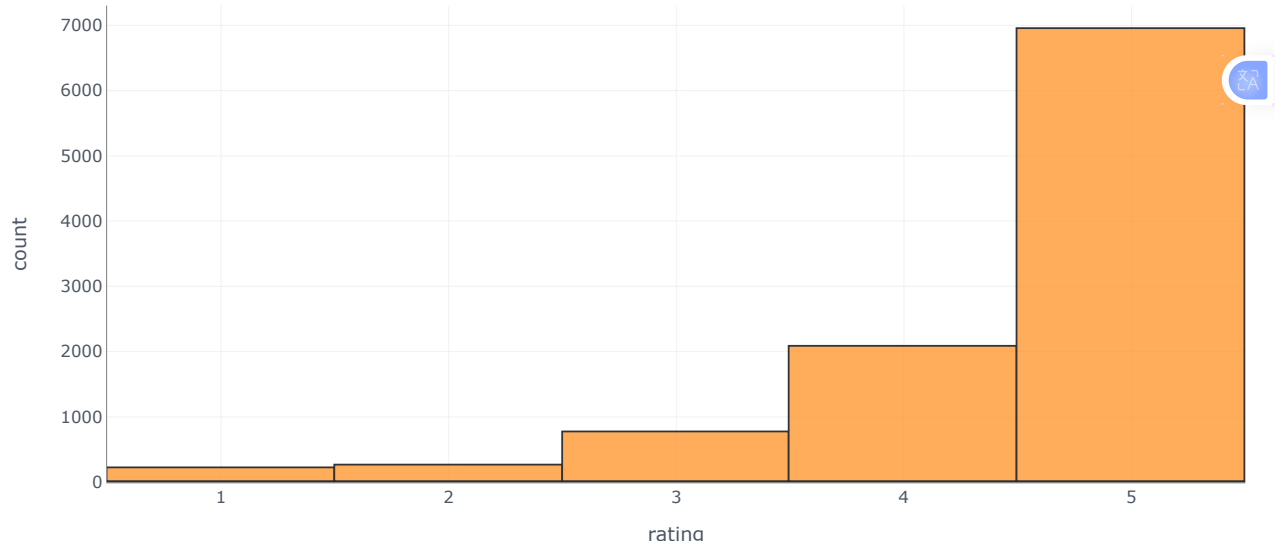
```
In [29]: process_reviews['polarity'].iplot(
    kind='hist',
    bins=50,
    xTitle='polarity',
    linecolor='black',
    yTitle='count',
    title='Sentiment Polarity Distribution')
```

Sentiment Polarity Distribution




```
In [30]: process_reviews['overall'].iplot(
        kind='hist',
        xTitle='rating',
        linecolor='black',
        yTitle='count',
        title='Review Rating Distribution')
```

Review Rating Distribution



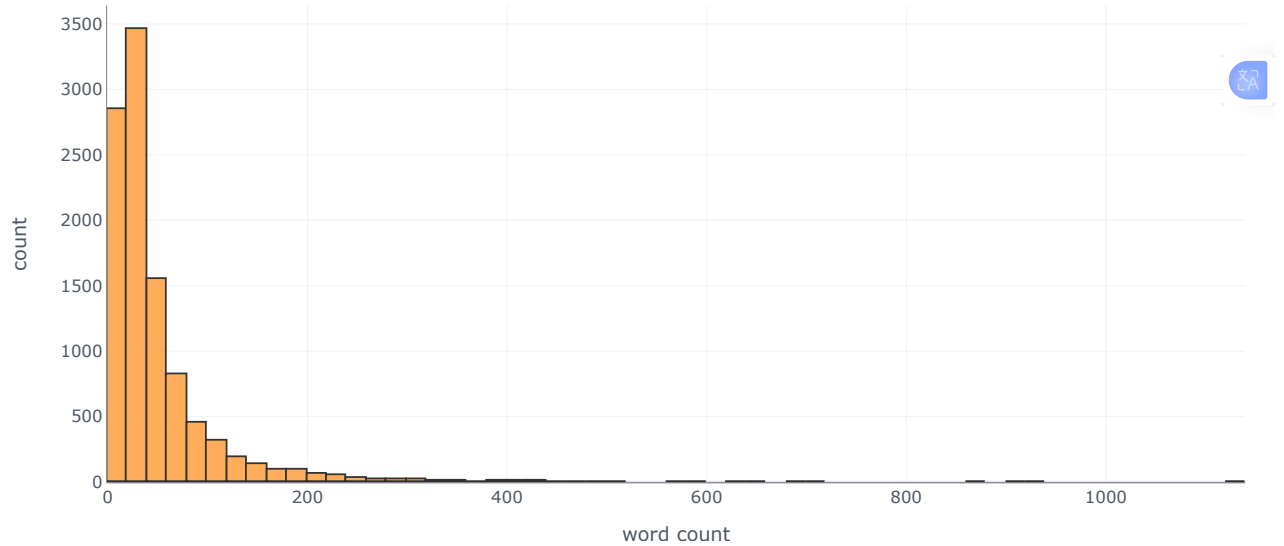
```
In [31]: process_reviews['review_len'].iplot(
        kind='hist',
        bins=100,
        xTitle='review length',
        linecolor='black',
        yTitle='count',
        title='Review Text Length Distribution')
```

Review Text Length Distribution



```
In [32]: process_reviews['word_count'].plot(  
    kind='hist',  
    bins=100,  
    xTitle='word count',  
    linecolor='black',  
    yTitle='count',  
    title='Review Text Word Count Distribution')
```

Review Text Word Count Distribution



```

In [33]: #Filtering data
review_pos = process_reviews[process_reviews["sentiment"]=="Positive"].dropna()
review_neu = process_reviews[process_reviews["sentiment"]=="Neutral"].dropna()
review_neg = process_reviews[process_reviews["sentiment"]=="Negative"].dropna()

## custom function for ngram generation ##
def generate_ngrams(text, n_gram=1):
    token = [token for token in text.lower().split(" ") if token != "" if token not in STOPWORDS]
    ngrams = zip(*[token[i:] for i in range(n_gram)])
    return [" ".join(ngram) for ngram in ngrams]

## custom function for horizontal bar chart ##
def horizontal_bar_chart(df, color):
    trace = go.Bar(
        y=df["word"].values[::-1],
        x=df["wordcount"].values[::-1],
        showlegend=False,
        orientation = 'h',
        marker=dict(
            color=color,
        ),
    )
    return trace

## Get the bar chart from positive reviews ##
freq_dict = defaultdict(int)
for sent in review_pos["reviews"]:
    for word in generate_ngrams(sent):
        freq_dict[word] += 1
fd_sorted = pd.DataFrame(sorted(freq_dict.items(), key=lambda x: x[1])[::-1])
fd_sorted.columns = ["word", "wordcount"]
trace0 = horizontal_bar_chart(fd_sorted.head(25), 'green')

## Get the bar chart from neutral reviews ##
freq_dict = defaultdict(int)
for sent in review_neu["reviews"]:
    for word in generate_ngrams(sent):
        freq_dict[word] += 1
fd_sorted = pd.DataFrame(sorted(freq_dict.items(), key=lambda x: x[1])[::-1])
fd_sorted.columns = ["word", "wordcount"]
trace1 = horizontal_bar_chart(fd_sorted.head(25), 'grey')

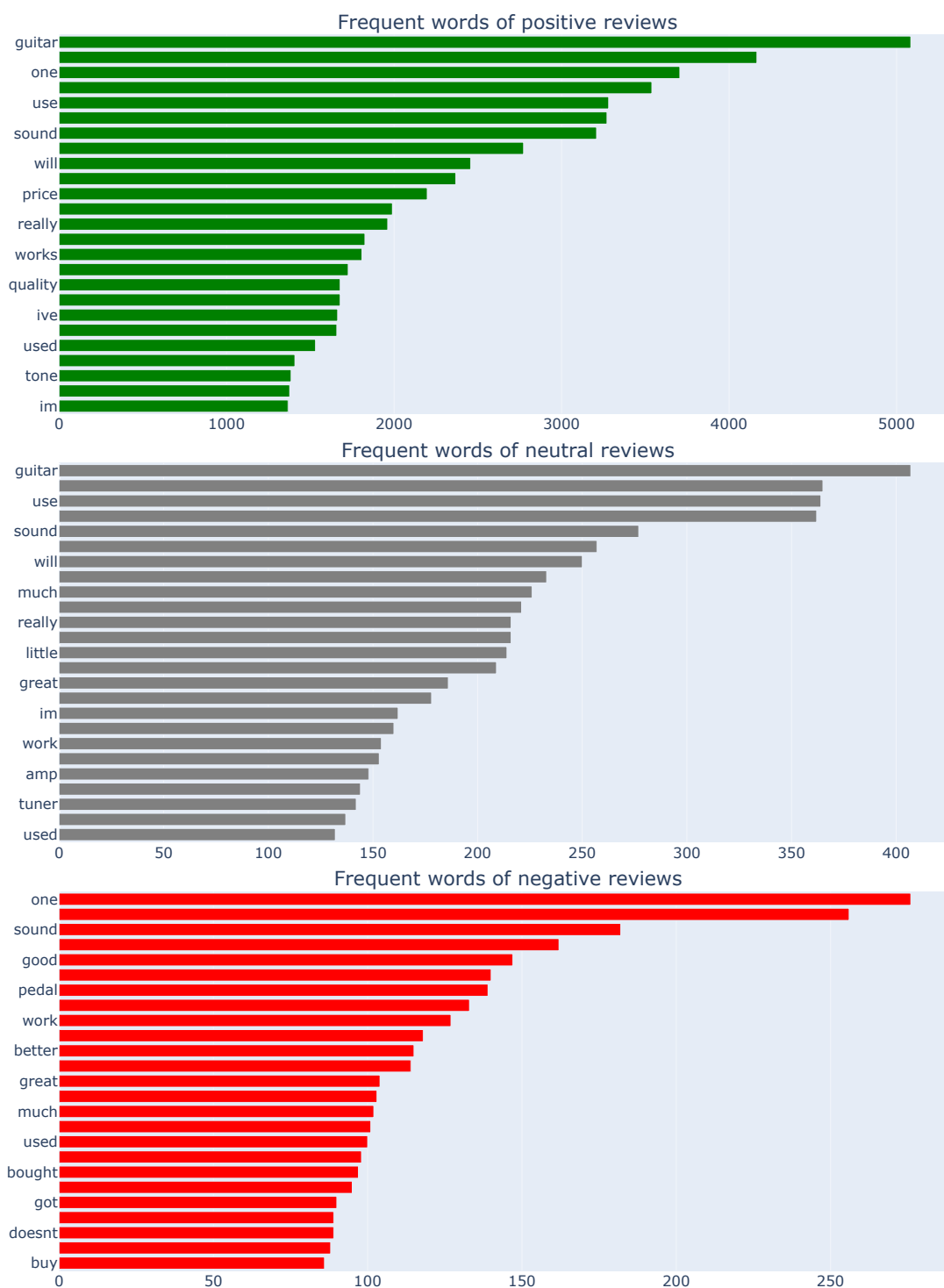
## Get the bar chart from negative reviews ##
freq_dict = defaultdict(int)
for sent in review_neg["reviews"]:
    for word in generate_ngrams(sent):
        freq_dict[word] += 1
fd_sorted = pd.DataFrame(sorted(freq_dict.items(), key=lambda x: x[1])[::-1])
fd_sorted.columns = ["word", "wordcount"]
trace2 = horizontal_bar_chart(fd_sorted.head(25), 'red')

# Creating two subplots
fig = tools.make_subplots(rows=3, cols=1, vertical_spacing=0.04,
                           subplot_titles=["Frequent words of positive reviews", "Frequent words of neutral reviews",
                                             "Frequent words of negative reviews"])

fig.append_trace(trace0, 1, 1)
fig.append_trace(trace1, 2, 1)
fig.append_trace(trace2, 3, 1)
fig['layout'].update(height=1200, width=900, paper_bgcolor='rgb(233,233,233)', title="Word Count Plots")
iplot(fig, filename='word-plots')

```

Word Count Plots



```

In [34]: ## Get the bar chart from positive reviews ##
freq_dict = defaultdict(int)
for sent in review_pos["reviews"]:
    for word in generate_ngrams(sent,2):
        freq_dict[word] += 1
fd_sorted = pd.DataFrame(sorted(freq_dict.items(), key=lambda x: x[1])[:-1])
fd_sorted.columns = ["word", "wordcount"]
trace0 = horizontal_bar_chart(fd_sorted.head(25), 'green')

## Get the bar chart from neutral reviews ##
freq_dict = defaultdict(int)
for sent in review_neu["reviews"]:
    for word in generate_ngrams(sent,2):
        freq_dict[word] += 1
fd_sorted = pd.DataFrame(sorted(freq_dict.items(), key=lambda x: x[1])[:-1])
fd_sorted.columns = ["word", "wordcount"]
trace1 = horizontal_bar_chart(fd_sorted.head(25), 'grey')

## Get the bar chart from negative reviews ##
freq_dict = defaultdict(int)
for sent in review_neg["reviews"]:
    for word in generate_ngrams(sent,2):
        freq_dict[word] += 1
fd_sorted = pd.DataFrame(sorted(freq_dict.items(), key=lambda x: x[1])[:-1])
fd_sorted.columns = ["word", "wordcount"]
trace2 = horizontal_bar_chart(fd_sorted.head(25), 'brown')

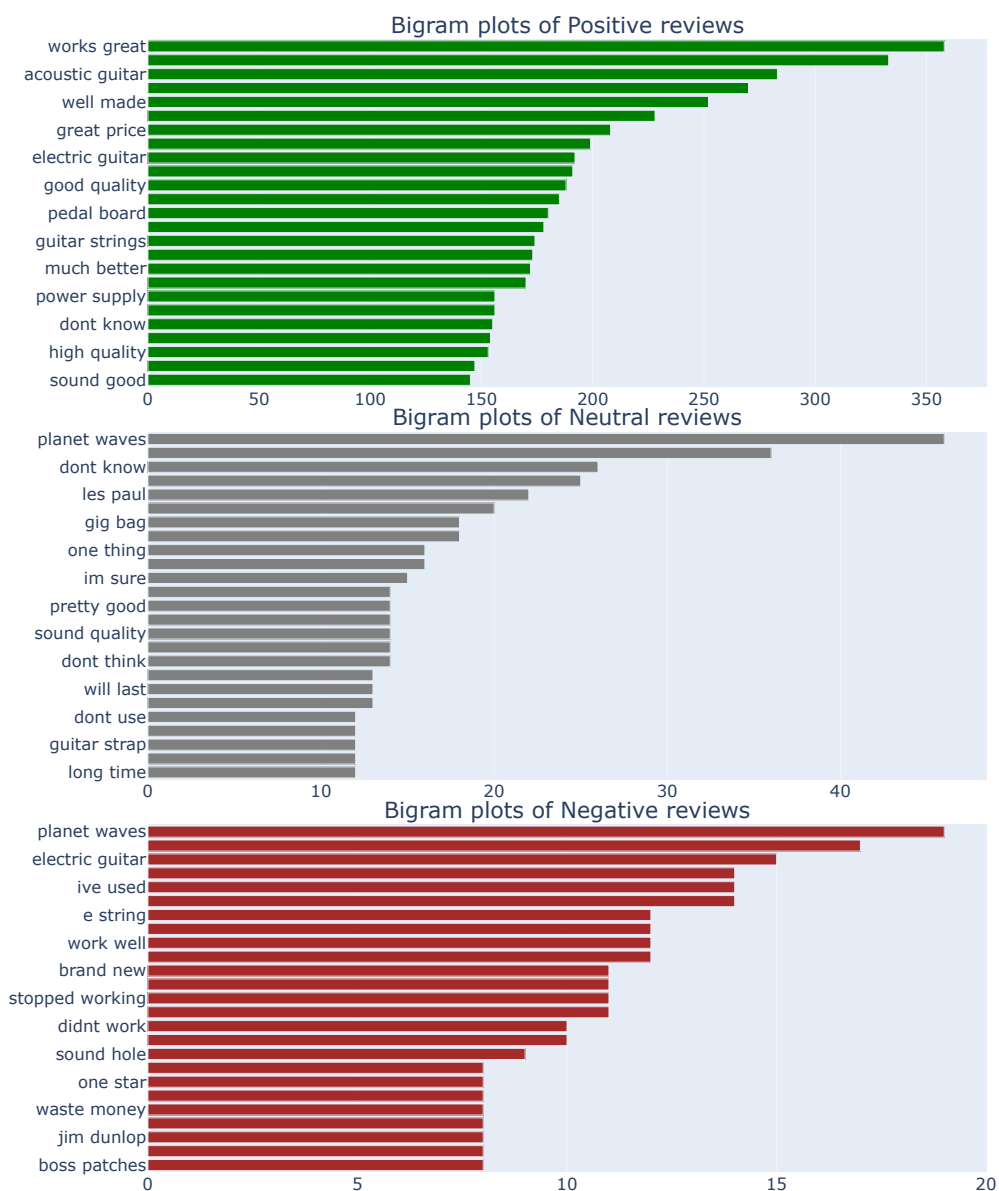
# Creating two subplots
fig = tools.make_subplots(rows=3, cols=1, vertical_spacing=0.04, horizontal_spacing=0.25,
                          subplot_titles=["Bigram plots of Positive reviews",
                                           "Bigram plots of Neutral reviews",
                                           "Bigram plots of Negative reviews"]
)

fig.append_trace(trace0, 1, 1)
fig.append_trace(trace1, 2, 1)
fig.append_trace(trace2, 3, 1)

fig['layout'].update(height=1000, width=800, paper_bgcolor='rgb(233,233,233)', title="Bigram Plots")
iplot(fig, filename='word-plots')

```

Bigram Plots



```

In [35]: ## Get the bar chart from positive reviews ##
for sent in review_pos["reviews"]:
    for word in generate_ngrams(sent,3):
        freq_dict[word] += 1
fd_sorted = pd.DataFrame(sorted(freq_dict.items(), key=lambda x: x[1])[:-1])
fd_sorted.columns = ["word", "wordcount"]
trace0 = horizontal_bar_chart(fd_sorted.head(25), 'green')

## Get the bar chart from neutral reviews ##
freq_dict = defaultdict(int)
for sent in review_neu["reviews"]:
    for word in generate_ngrams(sent,3):
        freq_dict[word] += 1
fd_sorted = pd.DataFrame(sorted(freq_dict.items(), key=lambda x: x[1])[:-1])
fd_sorted.columns = ["word", "wordcount"]
trace1 = horizontal_bar_chart(fd_sorted.head(25), 'grey')

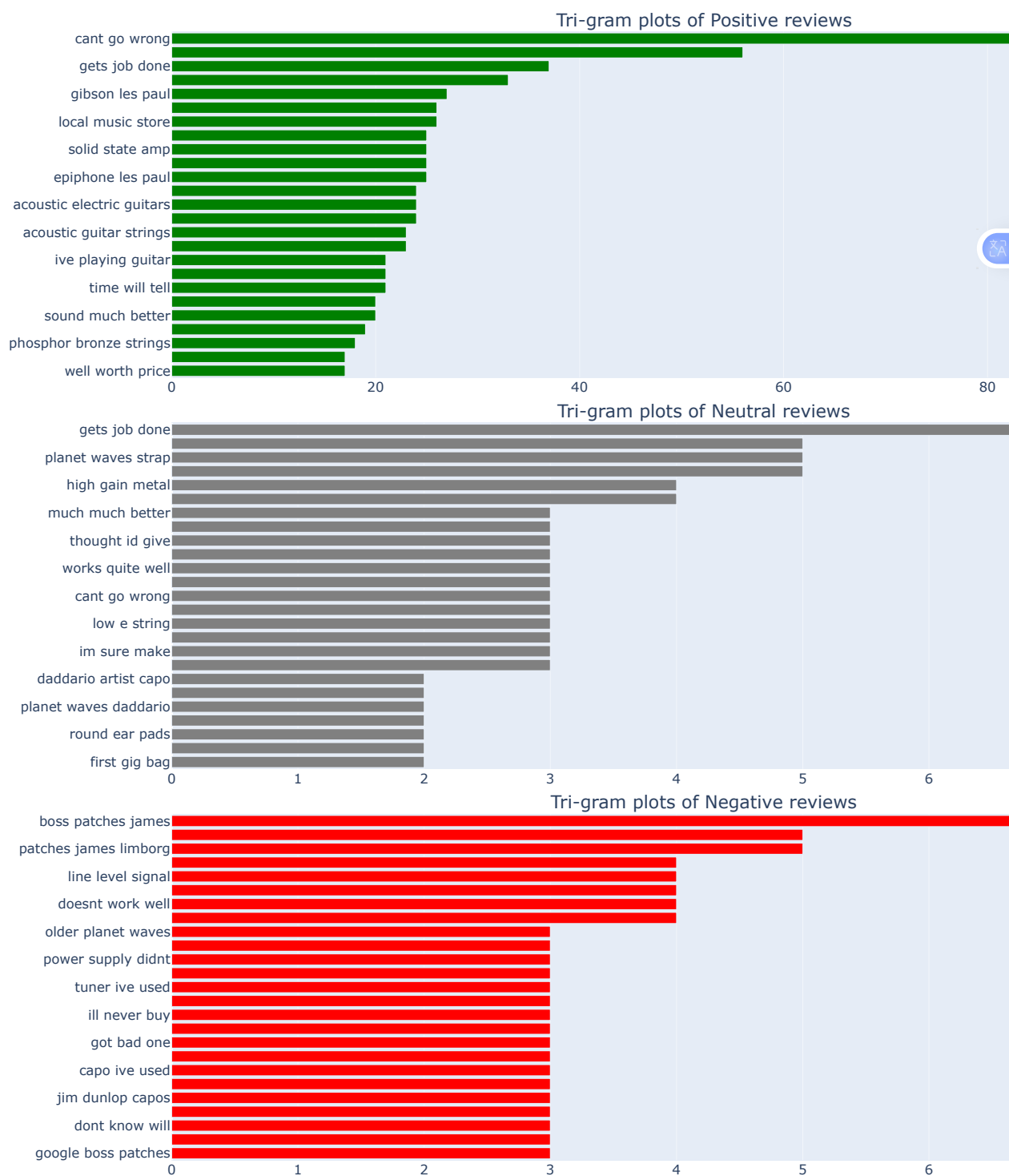
## Get the bar chart from negative reviews ##
freq_dict = defaultdict(int)
for sent in review_neg["reviews"]:
    for word in generate_ngrams(sent,3):
        freq_dict[word] += 1
fd_sorted = pd.DataFrame(sorted(freq_dict.items(), key=lambda x: x[1])[:-1])
fd_sorted.columns = ["word", "wordcount"]
trace2 = horizontal_bar_chart(fd_sorted.head(25), 'red')

# Creating two subplots
fig = tools.make_subplots(rows=3, cols=1, vertical_spacing=0.04, horizontal_spacing=0.05,
                          subplot_titles=["Tri-gram plots of Positive reviews",
                                           "Tri-gram plots of Neutral reviews",
                                           "Tri-gram plots of Negative reviews"])

fig.append_trace(trace0, 1, 1)
fig.append_trace(trace1, 2, 1)
fig.append_trace(trace2, 3, 1)
fig['layout'].update(height=1200, width=1200, paper_bgcolor='rgb(233,233,233)', title="Trigram Count Plots")
iplot(fig, filename='word-plots')

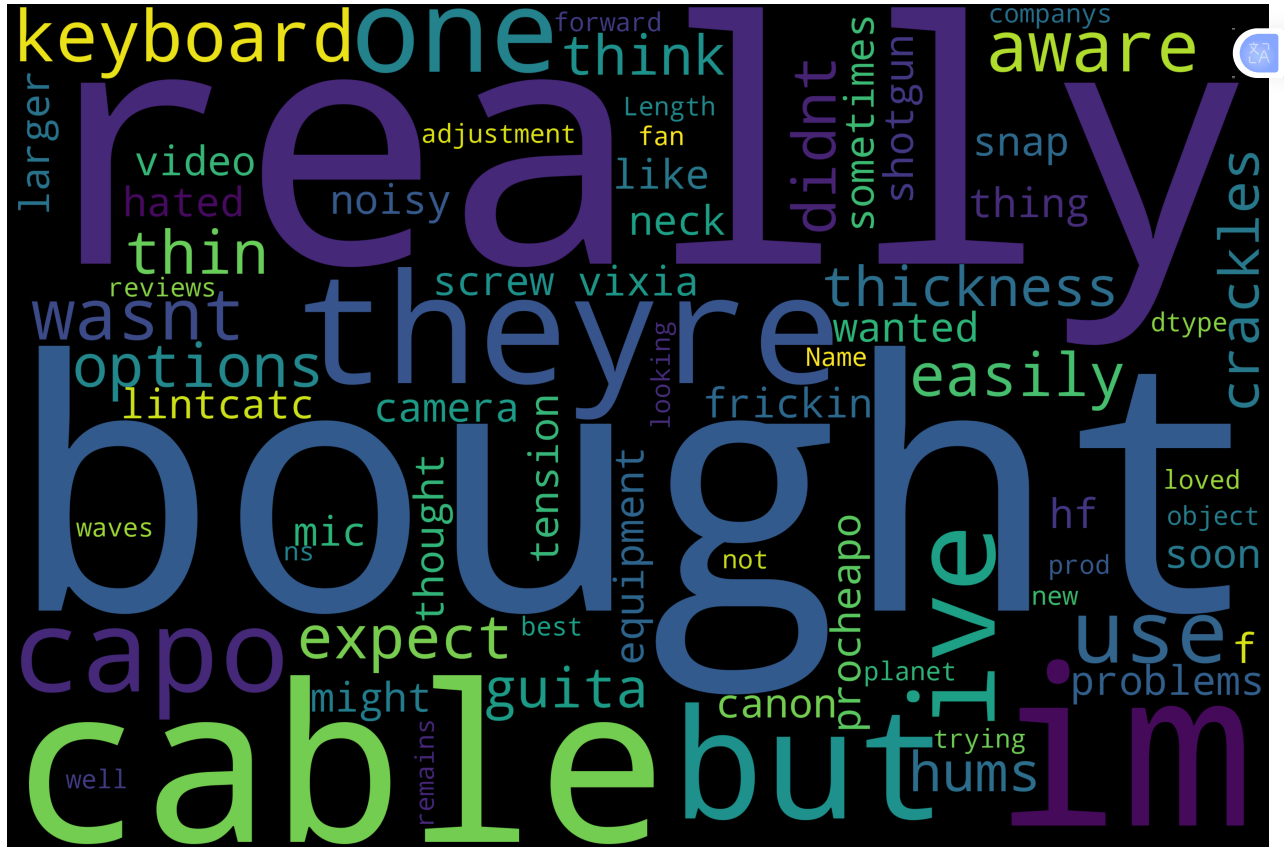
```

Trigram Count Plots



[illegible]


```
In [38]: text = review_neg["reviews"]
wordcloud = WordCloud(
    width = 3000,
    height = 2000,
    background_color = 'black',
    stopwords = stop_words).generate(str(text))
fig = plt.figure(
    figsize = (40, 30),
    facecolor = 'k',
    edgecolor = 'k')
plt.imshow(wordcloud, interpolation = 'bilinear')
plt.axis('off')
plt.tight_layout(pad=0)
plt.show()
```



```
In [39]: # calling the Label encoder function
label_encoder = preprocessing.LabelEncoder()

# Encode Labels in column 'sentiment'.
process_reviews['sentiment'] = label_encoder.fit_transform(process_reviews['sentiment'])

process_reviews['sentiment'].unique()
```

Out[39]: array([2, 1, 0])

```
In [40]: process_reviews['sentiment'].value_counts()
```

```
Out[40]: sentiment
2      9022
1       772
0       467
Name: count, dtype: int64
```

```
In [41]: #Extracting 'reviews' for processing
review_features=process_reviews.copy()
review_features=review_features[['reviews']].reset_index(drop=True)
review_features.head()
```

```
Out[41]:
```

	reviews
0	not much write but exactly supposed filters po...
1	product exactly quite affordablei not realized...
2	primary job device block breath would otherwis...
3	nice windscreen protects mxl mic prevents pops...
4	pop filter great looks performs like studio fi...

```
In [42]: #Performing stemming on the review dataframe
ps = PorterStemmer()

#splitting and adding the stemmed words except stopwords
corpus = []
for i in range(0, len(review_features)):
    review = re.sub('[^a-zA-Z]', ' ', review_features['reviews'][i])
    review = review.split()
    review = [ps.stem(word) for word in review if not word in stop_words]
    review = ' '.join(review)
    corpus.append(review)
```

```
In [43]: corpus[3]
```

```
Out[43]: 'nice windscreen protect mxl mic prevent pop thing gooseneck margin abl hold screen posit requir care posit clamp avoi
d sagginggood windscreen money'
```

```
In [44]: tfidf_vectorizer = TfidfVectorizer(max_features=5000,ngram_range=(2,2))
# TF-IDF feature matrix
X= tfidf_vectorizer.fit_transform(review_features['reviews'])
```

```
In [45]: X.shape
```

```
Out[45]: (10261, 5000)
```

```
In [46]: #Getting the target variable(encoded)
y=process_reviews['sentiment']
```

```
In [47]: import os
os.environ["OMP_NUM_THREADS"] = "1"
os.environ["MKL_NUM_THREADS"] = "1"

from collections import Counter
from imblearn.over_sampling import SMOTE

print(f'Original dataset shape : {Counter(y)}')

smote = SMOTE(random_state=42)
X_res, y_res = smote.fit_resample(X, y)

print(f'Resampled dataset shape : {Counter(y_res)}')
```

```
Original dataset shape : Counter({2: 9022, 1: 772, 0: 467})
```

```
-----
AttributeError                                Traceback (most recent call last)
```

```
Cell In[47], line 11
      8 print(f'Original dataset shape : {Counter(y)}')
     10 smote = SMOTE(random_state=42)
--> 11 X_res, y_res = smote.fit_resample(X, y)
     13 print(f'Resampled dataset shape : {Counter(y_res)}')
```

```
File C:\ProgramData\anaconda3\Lib\site-packages\imblearn\base.py:203, in BaseSampler.fit_resample(self, X, y)
    182 """Resample the dataset.
    183
    184 Parameters
    (...)
    200     The corresponding label of `X_resampled`.
    201 """
    202 self._validate_params()
--> 203 return super().fit_resample(X, y)
```

```
In [ ]: ## Divide the dataset into Train and Test
X_train, X_test, y_train, y_test = train_test_split(X_res, y_res, test_size=0.25, random_state=0)
```

```
In [ ]: def plot_confusion_matrix(cm, classes,
                                normalize=False,
                                title='Confusion matrix',
                                cmap=plt.cm.Blues):

    """
    This function prints and plots the confusion matrix.
    Normalization can be applied by setting `normalize=True`.
    """

    plt.imshow(cm, interpolation='nearest', cmap=cmap)
    plt.title(title)
    plt.colorbar()
    tick_marks = np.arange(len(classes))
    plt.xticks(tick_marks, classes, rotation=45)
    plt.yticks(tick_marks, classes)

    if normalize:
        cm = cm.astype('float') / cm.sum(axis=1)[:, np.newaxis]
        print("Normalized confusion matrix")
    else:
        print('Confusion matrix, without normalization')

    thresh = cm.max() / 2.
    for i in range(cm.shape[0]):
        for j in range(cm.shape[1]):
            plt.text(j, i, cm[i, j],
                    horizontalalignment="center",
                    color="white" if cm[i, j] > thresh else "black")

    plt.tight_layout()
    plt.ylabel('True label')
    plt.xlabel('Predicted label')
```

```
In [ ]: #creating the objects
logreg_cv = LogisticRegression(random_state=0)
dt_cv=DecisionTreeClassifier()
knn_cv=KNeighborsClassifier()
svc_cv=SVC()
nb_cv=BernoulliNB()
cv_dict = {0: 'Logistic Regression', 1: 'Decision Tree', 2: 'KNN', 3: 'SVC', 4: 'Naive Bayes'}
cv_models=[logreg_cv, dt_cv, knn_cv, svc_cv, nb_cv]

for i, model in enumerate(cv_models):
    print("{} Test Accuracy: {}".format(cv_dict[i], cross_val_score(model, X, y, cv=10, scoring='accuracy').mean()))
```

```
In [ ]: param_grid = {'C': np.logspace(-4, 4, 50),
                      'penalty': ['l1', 'l2']}
clf = GridSearchCV(LogisticRegression(random_state=0), param_grid, cv=5, verbose=0, n_jobs=-1)
best_model = clf.fit(X_train, y_train)
print(best_model.best_estimator_)
print("The mean accuracy of the model is:", best_model.score(X_test, y_test))
```

```
In [ ]: logreg = LogisticRegression(C=10000.0, random_state=0)
logreg.fit(X_train, y_train)
y_pred = logreg.predict(X_test)
print('Accuracy of logistic regression classifier on test set: {:.2f}'.format(logreg.score(X_test, y_test)))
```

```
In [ ]: cm = metrics.confusion_matrix(y_test, y_pred)
plot_confusion_matrix(cm, classes=['Negative', 'Neutral', 'Positive'])
```

```
In [ ]: print("Classification Report:\n", classification_report(y_test, y_pred))
```

```

In [ ]: #Binarizing the target feature
y = label_binarize(y, classes=[0, 1, 2])
n_classes = y.shape[1]

#Train-Test split(80:20)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=.2,
                                                    random_state=0)

#OneVsRestClassifier
classifier = OneVsRestClassifier(svm.SVC(kernel='linear', probability=True,
                                           random_state=10))
y_score = classifier.fit(X_train, y_train).decision_function(X_test)

#Computing TPR and FPR
fpr = dict()
tpr = dict()
roc_auc = dict()
for i in range(n_classes):
    fpr[i], tpr[i], _ = roc_curve(y_test[:, i], y_score[:, i])
    roc_auc[i] = auc(fpr[i], tpr[i])

# Compute micro-average ROC curve and ROC area
fpr["micro"], tpr["micro"], _ = roc_curve(y_test.ravel(), y_score.ravel())
roc_auc["micro"] = auc(fpr["micro"], tpr["micro"])

# aggregate all false positive rates
all_fpr = np.unique(np.concatenate([fpr[i] for i in range(n_classes)]))

# interpolate all ROC curves at this points
mean_tpr = np.zeros_like(all_fpr)
for i in range(n_classes):
    mean_tpr += interp(all_fpr, fpr[i], tpr[i])

# Finally average it and compute AUC
mean_tpr /= n_classes

fpr["macro"] = all_fpr
tpr["macro"] = mean_tpr
roc_auc["macro"] = auc(fpr["macro"], tpr["macro"])

# Plot all ROC curves
plt.figure()
plt.plot(fpr["micro"], tpr["micro"],
         label='micro-average ROC curve (area = {0:0.2f})'
         ''.format(roc_auc["micro"]),
         color='deeppink', linestyle=':', linewidth=4)

plt.plot(fpr["macro"], tpr["macro"],
         label='macro-average ROC curve (area = {0:0.2f})'
         ''.format(roc_auc["macro"]),
         color='navy', linestyle=':', linewidth=4)

colors = cycle(['aqua', 'darkorange', 'cornflowerblue'])
for i, color in zip(range(n_classes), colors):
    plt.plot(fpr[i], tpr[i], color=color, lw=4,
             label='ROC curve of class {0} (area = {1:0.2f})'
             ''.format(i, roc_auc[i]))

plt.plot([0, 1], [0, 1], 'k--', lw=4)
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver operating characteristic to multi-class')
plt.legend(loc="lower right")
plt.show()

```

In []:

In []: